NETCOM COMPUTERS PVT LTD • SOFTWARE ENGINEERING DIVISION

Enterprise Employee Management System Pro

Complete Technical Guide: Tools, Creation Workflow, Architecture, Functions & Methods, SQL, Django, Python, HTML5, CSS3 & JavaScript

Project Framework: Django 5.0.6 (MVT Architecture)	**Core Language:** Python 3.12+ (OOP & Clean Code)	**Database:** MySQL 8.0 & SQLite3 Multi-Engine
Frontend: Semantic HTML5 + CSS3 + ES6 JS	**Client APIs:** WebRTC MediaDevices & W3C GPS	**Documents:** ReportLab (PDF) & OpenPyXL (Excel)
Application Host: http://127.0.0.1:8000/login/	**Admin Credentials:** NETCOM / Netcom@2026@Netcom	**Scope:** HRMS, Attendance, Leave, Payroll, Audit

1. Development Tools & Software Environment

To build, develop, and run this enterprise employee management system from scratch, the following toolchain and software ecosystem are utilized:

Tool / Component	Category / Version	Exact Role & Implementation Purpose in Project
Python 3.12+	Programming Runtime	Core programming language powering the backend, data structures, date/time calculations, ORM logic, decorators, and file handling.
Django 5.0.6	Full-Stack Web Framework	Provides Model-View-Template (MVT) architecture, URL routing, ORM database engine, session auth, CSRF protection, and admin governance.
MySQL 8.0 / SQLite	Relational Database	ACID-compliant storage for users, staff records, attendance photos/GPS, leaves, and payroll records. SQLite is used for zero-config testing.
ReportLab 5.0.1	PDF Generation Engine	Programmatic vector PDF compilation for official documentation, employee dossiers, and attendance reports with headers and page counts.
OpenPyXL	Excel Generation Engine	Generates structured, multi-sheet corporate .xlsx spreadsheets with styled table headers, thin borders, and numeric formatting.
Pillow (PIL)	Image Processing	Validates, resizes, and processes employee profile photos, attendance verification selfies, banners, and offsite work proofs.
WhiteNoise & Gunicorn	WSGI Server & Static Asset Host	Gunicorn serves concurrent WSGI HTTP workers in production; WhiteNoise enables direct, high-speed static asset caching without Nginx.
Modern Web Browser	Chrome / Edge / Firefox	Provides runtime client APIs: navigator.mediaDevices.getUserMedia() (Camera) and navigator.geolocation (GPS).

2. Step-by-Step Project Creation Workflow

Follow these exact sequential steps and terminal commands to create and configure the entire codebase from scratch:

- **Step 1: Environment & Virtualenv Setup**

```
mkdir employee_management && cd employee_management
python -m venv venv
# Windows activation: venv\Scripts\activate  |  Linux/Mac: source venv/bin/activate
```

- **Step 2: Install Required Dependencies**

```
pip install django==5.0.6 reportlab==5.0.1 openpyxl pillow whitenoise gunicorn mysqlclient python-dotenv
pip freeze > requirements.txt
```

- **Step 3: Initialize Django Project & Modular Apps**

```
django-admin startproject config .
python manage.py startapp apps/employees
python manage.py startapp apps/departments
python manage.py startapp apps/attendance
python manage.py startapp apps/leave_management
python manage.py startapp apps/payroll
python manage.py startapp apps/dashboard
python manage.py startapp apps/reports
python manage.py startapp apps/users
```

- **Step 4: Configure config/settings.py**

```
# In INSTALLED_APPS add: 'apps.users', 'apps.employees', 'apps.departments',
# 'apps.attendance', 'apps.leave_management', 'apps.payroll', 'apps.dashboard', 'apps.reports'
# Configure TEMPLATES DIRS: [BASE_DIR / 'templates']
# Configure STATIC_URL = '/static/', STATICFILES_DIRS = [BASE_DIR / 'static'], STATIC_ROOT = BASE_DIR / 'staticfiles
'
# Configure MEDIA_URL = '/media/', MEDIA_ROOT = BASE_DIR / 'media'
```

- **Step 5: Database Migrations & Initial Setup**

```
python manage.py makemigrations users employees departments attendance leave_management payroll dashboard
python manage.py migrate
python manage.py createsuperuser  # Create main Admin account
```

- **Step 6: Launch Development Server**

```
python manage.py runserver 0.0.0.0:8000
# Navigate browser to: http://127.0.0.1:8000/login/
```

3. Relational Database Design & SQL Architecture

The system utilizes a fully normalized relational schema with primary keys, foreign key constraints with CASCADE rules, unique composite indexes, and strict data typing. Below are the key SQL tables, DDL structures, and relationships:

SQL Table Name	Key Columns & Data Types	Constraints & Relational Purpose
auth_user ***(Django Core)***	id INT AUTO_INCREMENT PRIMARY KEY username VARCHAR(150) UNIQUE password VARCHAR(128) first_name, last_name, email is_staff, is_superuser, is_active	Central identity table. Uses PBKDF2 SHA256 hashed passwords and session authentication tokens.
employees_employee	id INT AUTO_INCREMENT PRIMARY KEY user_id INT UNIQUE (FK auth_user) employee_code VARCHAR(20) UNIQUE role VARCHAR(20) [ADMIN, HR, EMPLOYEE] joining_date DATE NOT NULL phone VARCHAR(15), basic_salary DECIMAL(10,2) casual_leave_balance INT DEFAULT 1 raw_password VARCHAR(128) profile_picture VARCHAR(100), document VARCHAR(100)	One-to-One extension of auth_user. Mandatory joining_date isolates calendar and attendance calculations.
attendance_attendance	id INT AUTO_INCREMENT PRIMARY KEY employee_id INT (FK employees_employee) date DATE NOT NULL in_time TIME, out_time TIME status VARCHAR(20) [Present, Half Day, Absent] in_photo VARCHAR(255), out_photo VARCHAR(255) in_latitude DECIMAL(9,6), in_longitude DECIMAL(9,6) total_work_minutes INT DEFAULT 0 is_permission BOOLEAN, permission_hours DECIMAL(4,2)	**UNIQUE KEY (employee_id, date)**. Strictly enforces single daily punch session with photo and GPS tracking.
attendance_attendanceauditlog	id INT AUTO_INCREMENT PRIMARY KEY attendance_id INT (FK attendance_attendance) changed_by_id INT (FK auth_user) action VARCHAR(50), timestamp DATETIME old_values JSON/TEXT, new_values JSON/TEXT ip_address VARCHAR(45)	Immutable audit trail tracking every manual override, missing punch correction, and administrative adjustment.
leave_management_leaverequest	id INT AUTO_INCREMENT PRIMARY KEY employee_id INT (FK employees_employee) leave_type VARCHAR(20) [Casual, Sick, LWP, Permission] from_date DATE, to_date DATE, total_days INT status VARCHAR(20) [Pending, Approved, Rejected] reason TEXT, is_paid BOOLEAN, reviewed_by_id INT	Leave applications and 2-hour permission submissions. Approved paid leaves credit attendance with zero salary deduction.
leave_management_monthlyleavecredit	id INT AUTO_INCREMENT PRIMARY KEY employee_id INT (FK employees_employee) year INT, month INT, casual_added INT DEFAULT 1 created_at DATETIME	**UNIQUE KEY (employee_id, year, month)**. Strictly enforces the company rule of 1 casual leave credit per month without rollover.
payroll_salarystructure	id INT AUTO_INCREMENT PRIMARY KEY employee_id INT UNIQUE (FK employees_employee) basic_salary DECIMAL(10,2) hra DECIMAL(10,2), allowances DECIMAL(10,2) statutory_deductions DECIMAL(10,2)	Base salary blueprint for each staff member. Gross Salary = Basic + HRA + Allowances.
payroll_payrollrecord	id INT AUTO_INCREMENT PRIMARY KEY employee_id INT (FK employees_employee) year INT, month INT, gross_salary DECIMAL(10,2) attendance_deductions DECIMAL(10,2) net_salary DECIMAL(10,2), status VARCHAR(20)	**UNIQUE KEY (employee_id, year, month)**. Monthly payslip record with computed daily rates and attendance deductions.

SQL DDL & ORM Query Translation Examples:

```
-- 1. SQL DDL Table Creation for Attendance Record
CREATE TABLE attendance_attendance (
    id INT AUTO_INCREMENT PRIMARY KEY,
    employee_id INT NOT NULL,
    date DATE NOT NULL,
    in_time TIME NULL, out_time TIME NULL,
    status VARCHAR(20) NOT NULL DEFAULT 'Present',
    in_photo VARCHAR(255) NULL, out_photo VARCHAR(255) NULL,
    in_latitude DECIMAL(9,6) NULL, in_longitude DECIMAL(9,6) NULL,
    total_work_minutes INT NOT NULL DEFAULT 0,
    CONSTRAINT fk_attendance_employee FOREIGN KEY (employee_id) REFERENCES employees_employee(id) ON DELETE CASCADE,
    CONSTRAINT uq_attendance_emp_date UNIQUE (employee_id, date)
);

-- 2. Equivalent High-Performance Django ORM Query with Joins:
qs = Attendance.objects.filter(date__year=2026, date__month=10).select_related('employee__user', 'employee__departme
nt') \
        .annotate(total_punches=Count('id')).order_by('-date')
```

4. Backend Architecture: Python & Django Functions and Methods

Every core application module contains dedicated function-based views (FBVs), model methods, custom decorators, and business logic routines. Below is the comprehensive technical catalog of all major functions and methods:

Module / App	Function / Method Signature	Parameters, Internal Algorithm & Return Value
apps.users ***Authentication***	login_view(request)	**Args:** request (POST/GET). **Logic:** Validates credentials via authenticate(). If valid, logs user in via login() and routes dynamically to /dashboard/ based on role (Admin, HR, Employee). **Returns:** HTTP 302 Redirect or rendered users/login.html.
apps.users ***Authentication***	logout_view(request)	**Logic:** Calls request.session.flush() and django.contrib.auth.logout(), invalidating user session cookies to prevent session hijacking. **Returns:** Redirects to /login/.
apps.users ***User Profile***	user_profile_view(request)	**Logic:** Allows users to view profile, upload avatar and documents. If password is changed, updates both Django password hash and employee.raw_password for HR visibility. **Returns:** Rendered users/profile.html.
apps.users ***Security Decorator***	@admin_required @hr_or_admin_required	**Logic:** Inspects request.user.employee.role and is_superuser. Raises PermissionDenied (HTTP 403) or redirects to login if user lacks authorization.
apps.users ***Self-Healing***	get_or_create_employee_profile(user)	**Logic:** Auto-creates or heals an Employee profile for users lacking one, mapping superusers to ADMIN and 'hr' users to HR role automatically.
apps.employees ***Workforce Master***	employee_create(request)	**Logic:** Creates auth_user, generates unique employee_code, assigns department/designation, captures mandatory joining_date, and saves initial salary and raw password. **Returns:** Redirect to staff directory.
apps.employees ***Workforce Master***	quick_edit_joining_date(request)	**Logic:** AJAX/POST endpoint allowing HR/Admin to immediately update Employee.joining_date. Triggers automatic refresh of active attendance calendars and payroll calculations.
apps.employees ***Model Method***	Employee.is_eligible_for_date(date)	**Logic:** Evaluates if date >= self.joining_date. Returns False for prior dates, isolating pre-joining days from absenteeism, missed punch penalties, and salary cuts.
apps.attendance ***Duty Punch***	punch_in(request)	**Logic:** Decodes Base64 live selfie camera photo, captures W3C GPS latitude/longitude, verifies once-per-day rule, and creates Attendance record with in_time=now(). **Returns:** JSON success response.
apps.attendance ***Duty Punch***	punch_out(request)	**Logic:** Saves exit webcam selfie photo, records out_time, calculates total_work_minutes, and evaluates status: **Present (≥9h)**, **Half Day (4.5h–9h)**, or **Absent (<4.5h)**.
apps.attendance ***Governance***	attendance_override(request, attendance_id)	**Logic:** Allows HR/Admin to adjust punch times and status. Automatically logs old vs new values, user ID, timestamp, and IP address into AttendanceAuditLog for compliance.
apps.attendance ***Task Log***	daily_work_submit(request)	**Logic:** Captures employee's completed tasks, hours worked, on-site/off-site flag, and optional off-site photo proof into DailyWork. **Returns:** Redirect to dashboard.
apps.leave_management ***Leave Workflow***	apply_leave(request)	**Logic:** Validates dates, checks available casual_leave_balance, prevents overlapping requests, and creates LeaveRequest with status='Pending'.
apps.leave_management ***Leave Review***	review_leave(request, leave_id)	**Logic:** HR approves or rejects leave. If approved as Paid Casual Leave, deducts 1 day from balance and syncs calendar with full wage credit (zero salary deduction).
apps.leave_management ***Permission Workflow***	permission_request(request)	**Logic:** Validates sub-day 2-hour permission requests against monthly 5-hour quota. Prevents exceeding balance and sends request to HR review queue.
apps.leave_management ***Quota Engine***	credit_monthly_casual_leave()	**Logic:** Automated 1st of month cron/signal routine that credits exactly 1 casual leave to active employees, enforcing non-cumulative zero-rollover policy.
apps.payroll ***Payroll Engine***	calculate_daily_rate(gross, year, month)	**Formula:** gross_salary / calendar.monthrange(year, month)[1]. Calculates the exact daily rate based on total calendar days in billing month (28, 29, 30, or 31).
apps.payroll ***Payroll Engine***	calculate_attendance_deductions(emp, year, month)	**Formula:** (half_days × 0.5 × rate) + (absent_days × 1.0 × rate) + (sandwich_days × 1.0 × rate). Excludes dates before joining date and approved permissions.
apps.payroll ***Batch Generation***	generate_monthly_payroll(request)	**Logic:** Iterates through all active staff, computes attendance deductions, derives Net Salary = Gross - Deductions, and persists records in PayrollRecord.

apps.reports ***Export Engine***	export_attendance_excel(request)	**Logic:** Uses openpyxl to build styled Excel spreadsheets with headers, borders, and date filters. **Returns:** application/vnd.openxmlformats-officedocument.spreadsheetml.sheet.
apps.reports ***Export Engine***	export_dossier_pdf(request, emp_id)	**Logic:** Uses reportlab to generate complete executive employee dossiers with embedded avatar, attendance history, and leave stats. **Returns:** application/pdf.

5. Frontend Architecture: HTML5, CSS3 & JavaScript (ES6+)

The user presentation layer is built without heavyweight JavaScript frameworks, utilizing modern semantic HTML5, a modular custom CSS3 design system, and vanilla ES6+ JavaScript for hardware camera access and GPS positioning.

5.1 Semantic HTML5 & Django Template Language (DTL)

• **Base Template Inheritance:** All templates extend templates/base.html using Django's {% extends 'base.html' %} and {% block content %}.
• **Global Context Injection:** Banners, branding logo, announcement popups, and user role badges are injected globally across all views via custom context processors.
• **Hardware Viewfinder Elements:** Modern HTML5 <video id='webcam-video' autoplay playsinline></video> and hidden <canvas id='webcam-canvas'></canvas> elements capture live video streams for photo punch-in.
• **Form Security & File Uploads:** All forms enforce CSRF tokens ({% csrf_token %}) and support file/document uploads using enctype='multipart/form-data'.

```
<!-- HTML5 Webcam Viewfinder & Duty Punch Terminal Component --><br/><div class="punch-terminal-card"><br/>    <div class="webcam-container"><br/>        <video id="webcam-video" autoplay playsinline muted></video><br/>        <canvas id="webcam-canvas" style="display: none;"></canvas><br/>        <div class="gps-badge" id="gps-display"><i class="fas fa-map-marker-alt"></i> Acquiring GPS...</div><br/>    </div><br/>    <div class="punch-controls"><br/>
<button type="button" class="btn btn-success" id="btn-punch-in"><i class="fas fa-sign-in-alt"></i> Punch In</button><br/>        <button type="button" class="btn btn-danger" id="btn-punch-out"><i class="fas fa-sign-out-alt"></i> Punch Out</button><br/>    </div><br/></div>
```

5.2 Custom CSS3 Design System & Responsive Layouts

• **CSS Variables (Theming):** Root stylesheet defines standard corporate colors: --primary-color: #1e3a8a;, --secondary-color: #0284c7;, --dark-slate: #0f172a;.
• **Flexbox & CSS Grid:** Dashboard layout uses CSS Grid (grid-template-columns: repeat(auto-fit, minmax(280px, 1fr));) for auto-adapting KPI cards across mobile, tablet, and widescreen monitors.
• **Attendance Calendar Badges:** Solid Green (Present, Holiday, Sunday), Solid Yellow (Half Day), Green with Yellow Border (Approved Permission), Solid Red (Absent / LWP), and Muted Gray (Pre-Joining Inactive).
• **4-Record & 5-Record Scroll Containers:** Embedded tables use constrained max-height containers (max-height: 280px; overflow-y: auto;) preventing page elongation.

```
/* Attendance Calendar High-Contrast Color System */
.calendar-day.present       { background-color: #059669; color: #ffffff; font-weight: bold; }
.calendar-day.half-day      { background-color: #eab308; color: #0f172a; font-weight: bold; }
.calendar-day.absent        { background-color: #dc2626; color: #ffffff; font-weight: bold; }
.calendar-day.permission    { background-color: #059669; border: 2.5px solid #facc15; color: #ffffff; }
.calendar-day.office-leave  { background-color: #7c3aed; color: #ffffff; font-weight: bold; }
.calendar-day.pre-joining   { background-color: #f1f5f9; color: #94a3b8; cursor: not-allowed; }
```

5.3 Client-Side JavaScript (ES6+): WebRTC Camera & Geolocation APIs

• **WebRTC Camera Stream Initialization:** Invokes navigator.mediaDevices.getUserMedia({ video: { facingMode: 'user' } }) to bind live hardware webcam video to the viewfinder.
• **Snapshot Frame Capture:** Draws current video frame onto hidden HTML5 canvas: canvas.getContext('2d').drawImage(video, 0, 0, 640, 480), converting image to Base64 PNG via canvas.toDataURL('image/png').
• **W3C GPS Geolocation:** Calls navigator.geolocation.getCurrentPosition() with enableHighAccuracy: true, timeout: 10000 to capture real-time latitude and longitude coordinates.
• **Asynchronous AJAX POST:** Submits captured photo data and GPS coordinates to /attendance/punch-in/ via fetch() API with X-CSRFToken header.
• **Password Visibility Toggle:** Interactive eye icon (■■) dynamically switches input between type='password' and type='text'.

```
// Live Camera Snapshot and GPS Attendance Submission (ES6+)
async function captureAndPunch(punchType) {
    const video = document.getElementById('webcam-video');
    const canvas = document.getElementById('webcam-canvas');
    canvas.width = 640; canvas.height = 480;
    canvas.getContext('2d').drawImage(video, 0, 0, 640, 480);
    const photoDataUrl = canvas.toDataURL('image/jpeg', 0.85);

    navigator.geolocation.getCurrentPosition(async (pos) => {
        const payload = {
            photo: photoDataUrl,
            latitude: pos.coords.latitude,
            longitude: pos.coords.longitude
        };
        const res = await fetch(`/attendance/punch-${punchType}/`, {
            method: 'POST',
            headers: { 'Content-Type': 'application/json', 'X-CSRFToken': getCsrfToken() },
```

```
            body: JSON.stringify(payload)
        });
        const data = await res.json();
        if (data.status === 'success') { window.location.reload(); }
        else { alert('Attendance Error: ' + data.message); }
    }, (err) => { alert('GPS Geolocation Error: Please enable location services.'); }, { enableHighAccuracy: true })
;
}
```

6. Project Code Structure & Execution Commands

Complete directory hierarchy and one-click execution commands for running the project on local, LAN, and production environments:

```
employee_management/
■■■ manage.py                  # Django CLI management executable
■■■ requirements.txt           # Python package dependencies
■■■ run_server.bat             # One-click Windows launcher (Local & LAN)
■■■ .env                       # Environment variables (SECRET_KEY, DB credentials)
■■■ config/                    # Django core project configuration
■   ■■■ settings.py            # Global settings, DB, static, middleware
■   ■■■ urls.py                # Root URL router
■   ■■■ wsgi.py                # WSGI application entry for Gunicorn
■■■ apps/                      # Modular business logic applications
■   ■■■ users/                 # Custom auth, login, profiles, context processors
■   ■■■ employees/             # Master staff registry, joining date, documents
■   ■■■ departments/           # Departments & designations
■   ■■■ attendance/            # GPS & photo punch, audit logs, daily work
■   ■■■ leave_management/      # Leave requests, 2-hr permissions, monthly credit
■   ■■■ payroll/               # Salary structure, attendance-based salary calculations
■   ■■■ dashboard/             # Role-based dashboards (Admin, HR, Employee)
■   ■■■ reports/               # Excel (.xlsx) and PDF export engine
■■■ templates/                 # Server-rendered HTML5 templates
■■■ static/                    # CSS stylesheets, JavaScript files, corporate images
■■■ media/                     # Uploaded staff documents, profile photos, punch proofs
■■■ database/                  # Schema SQL scripts and JSON backup fixtures
```

One-Click Launcher Script (run_server.bat):

```
@echo off
title NETCOM Employee Management Pro - Server Launcher
cd /d "%~dp0"
echo [*] Checking database migrations...
python manage.py migrate
echo [*] Launching application in browser...
start http://127.0.0.1:8000/login/
echo [*] Starting Server on http://0.0.0.0:8000/ (Local & LAN Access)
python manage.py runserver 0.0.0.0:8000
pause
```

End of Technical & Developer Reference Guide • Netcom Computers Pvt Ltd • All Rights Reserved